Reproduction and Analysis of Self-Supervised Poisson Denoising: The Poisson2Sparse Framework

Submitted in partial fulfillment of the
requirements for the degree

of

Bachelor of Science in Electrical and Electronics Engineering

by

Abdullah Altepe 150721034

Under the supervision of

Asst. Prof. Dr. Rıfat Volkan Şenyuva

MARMARA
UNIVERSITY

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

FACULTY OF ENGINEERING

MARMARA UNIVERSITY

January, 2026

DECLARATION OF AUTHORSHIP

Project Title	Reproduction and Analysis of Self-Supervised Poisson Denoising: The Poisson2Sparse Framework
Authors	*Abdullah Altepe*
Supervisor	Asst. Prof. Dr. Rıfat Volkan Şenyuva

I, *Abdullah Altepe*, hereby declare that this project titled *Reproduction and Analysis of Self-Supervised Poisson Denoising: The Poisson2Sparse Framework* is completely my own work. The outcome I have reached is the complete result of my work, and I have complied with all ethical guidelines and academic standards.

I am aware of the consequences of academic dishonesty and understand that any violation of ethical standards in this project may lead to disciplinary actions as defined by Marmara University's policies.

Abdullah Altepe
150721034

Istanbul
January, 2026

Electrical and Electronics Engineering
Engineering
Marmara University
Istanbul, Türkiye

Reproduction and Analysis of Self-Supervised Poisson Denoising: The Poisson2Sparse Framework

by

Abdullah Altepe

SUBMITTED TO THE DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR OF SCIENCE
AT
MARMARA UNIVERSITY

The author(s) hereby grant(s) to Marmara University permission to reproduce and to distribute publicly paper and electronic copies of this document in whole or in part and declare that the prepared document does not in any way include copying of previous work on the subject or the use of ideas, concepts, words, or structures regarding the subject without appropriate acknowledgement of the source material.

Signature of Author

Supervisor
Asst. Prof. Dr. Rıfat Volkan Şenyuva

Head of Department
Prof. Dr. Engin Maşazade

ACKNOWLEDGEMENTS

I would like to sincerely thank my thesis supervisor, Asst. Prof. Dr. Rıfat Volkan Şenyuva, for his guidance and steady support throughout this work. His feedback, both technical and practical, played a major role in shaping the direction of the project, especially at times when the reproduction of the Poisson2Sparse framework became challenging. His way of asking the right questions often helped me see problems from a clearer perspective.

I would like to acknowledge the authors of the original Poisson2Sparse paper for making their work and code publicly available. Without this level of openness, it would not have been possible to carry out an independent reproduction study. Their contribution gave me the opportunity to explore how self–supervised methods behave in practice, especially on hardware that is far from a typical research lab setup.

Abdullah Altepe
Marmara University
Date: January, 2026

ABSTRACT

Biomedical imaging techniques such as fluorescence microscopy and magnetic resonance imaging are strongly affected by Poisson noise, especially in low–signal conditions. In many medical applications, it is not possible to obtain a truly clean reference image, which makes standard supervised denoising methods difficult to apply. For this reason, self–supervised approaches that learn directly from noisy data have become increasingly important.

In this thesis, we focus on the Poisson2Sparse framework and provide both a detailed analysis and an independent reproduction of the method. Poisson2Sparse combines convolutional sparse coding with a deep unrolling strategy, allowing an iterative optimization algorithm to be expressed as a neural network. The main goal of this study is to examine how well this approach holds up when it is run outside of a high–performance computing environment.

The full model was re–implemented and trained on the PINCAT dataset using an Apple M4 system. Although this setup is very different from the CUDA–based GPU hardware used in the original work, the network was able to converge to a similar level of performance. In our experiments, the Peak Signal-to-Noise Ratio reached 31.88 dB (approaching the reference benchmark of 32.20 dB), which demonstrates the effectiveness with the results reported in the original study. In addition, the method noise maps showed that the network removes random Poisson noise while keeping the underlying anatomical structures intact.

Overall, this work confirms both the theoretical soundness and the practical usefulness of the Poisson2Sparse approach. It also shows that advanced self–supervised denoising methods can be run on consumer–grade hardware, making them more accessible for research and potentially for clinical applications.

Keywords: Self–Supervised Learning, Image Denoising, Poisson Noise, Sparse Coding, Deep Unrolling, Reproducibility, Biomedical Imaging.

ABBREVIATIONS

API	Application Programming Interface
CNN	Convolutional Neural Network
CSC	Convolutional Sparse Coding
CUDA	Compute Unified Device Architecture
DIP	Deep Image Prior
FMD	Fluorescence Microscopy Denoising (Dataset)
GPU	Graphics Processing Unit
ISTA	Iterative Shrinkage-Thresholding Algorithm
LISTA	Learned Iterative Shrinkage-Thresholding Algorithm
MPS	Metal Performance Shaders
MSE	Mean Squared Error
PLL	Poisson Log-Likelihood
PSNR	Peak Signal-to-Noise Ratio
ReLU	Rectified Linear Unit
SSIM	Structural Similarity Index Measure
UMA	Unified Memory Architecture

Contents

1 INTRODUCTION **1**

1.1 Thesis Background 2

1.2 Project Definition 3

1.2.1 Problem statement 3

1.2.2 Context and scope 3

1.2.3 Significance and implications 4

1.3 Technical Architecture Overview 4

1.3.1 Theoretical foundation 4

1.3.2 Model framework 5

1.3.3 Hardware and verification environment 5

1.4 Core Contributions 6

1.4.1 Independent reproduction 6

1.4.2 Hardware adaptation: from CUDA to MPS 6

1.5 Development Methodology 7

1.5.1 Framework selection 7

1.5.2 Verification strategy 7

2 RESEARCH OBJECTIVES **9**

2.1 Primary research objectives 9

2.1.1 Independent algorithmic reproduction 9

2.1.2 Cross-platform hardware verification (CUDA to MPS) 10

2.1.3 Robustness checks for self-supervised denoising 10

2.2 Project specifications 10

2.2.1 Hardware environment: Apple Silicon 11

2.2.2 Software stack and frameworks 11

2.2.3 Data specifications 11

2.3 Success criteria . . . 12
2.4 Functional requirements analysis . . . 12
2.4.1 Data processing pipeline . . . 13
2.4.2 Model training and optimization . . . 13
2.4.3 Inference and validation . . . 13
2.5 Technical constraints . . . 14
2.5.1 Floating-point precision and numerical stability . . . 14
2.5.2 Computational throughput and training latency . . . 14
2.5.3 Unified memory architecture (UMA) limitations . . . 15

3 RELATED LITERATURE 16
3.1 Traditional Denoising Approaches . . . 16
3.2 Supervised Deep Learning . . . 17
3.3 Self-Supervised Learning Paradigms . . . 17
3.4 Sparse Representation and Deep Unrolling . . . 18
3.5 Motivation for Selecting Poisson2Sparse . . . 18

4 SYSTEM DESIGN 20
4.1 Noise Characterization: Poisson Distribution . . . 20
4.1.1 Why Poisson? . . . 20
4.1.2 Mathematical Model of the Poisson Distribution . . . 20
4.1.3 Maximum Likelihood Estimation and Loss Function . . . 21
4.2 Sparse Representation Theory . . . 21
4.2.1 Core philosophy and modeling approach . . . 21
4.2.2 Convolutional sparse coding formulation . . . 22
4.3 Deep Algorithm Unrolling . . . 22
4.3.1 Deep Unrolling: From ISTA to a Neural Network . . . 22
4.3.1.1 Turning iterations into layers . . . 23
4.3.1.2 Learning instead of tuning . . . 24
4.3.2 Sparsity and Soft Thresholding . . . 24
4.3.2.1 Sparsity as a filtering principle . . . 25
4.3.2.2 Soft thresholding . . . 25
4.3.2.3 Iterative refinement . . . 25
4.4 Training Objective and Loss Functions . . . 26
4.4.1 Poisson loss . . . 26

4.4.2 Neighbor consistency regularization 27
4.4.3 Reconstruction stability 27

5 METHODS **28**
5.1 Data Preparation Strategy 28
5.1.1 The problem: lack of clean ground truth data 28
5.1.2 Neighbor down-sampling 28
5.2 Network Architecture Implementation 29
5.2.1 Network architecture 30
5.2.2 Hardware and runtime 30

6 RESULTS **31**
6.0.1 Qualitative results and noise analysis 31
6.1 Quantitative Evaluation 32
6.1.1 Metric Definition and Experimental Protocol 32
6.1.2 Robustness to Varying Noise Levels 32
6.1.3 Comparison with Baseline Results 34
6.2 Hyperparameter Sensitivity Analysis 35
6.3 Analysis of Learned Features 35

7 CONCLUSION **38**
7.1 Summary of findings 38
7.2 Hardware portability and accessibility 39
7.3 Future work 39

REFERENCES **40**

List of Figures

4.1 Iterative Image Refinement Process over 10 steps 23
4.2 Visualization of the Soft Thresholding Operator (S_τ) 24

5.1 Neighbor Down-Sampling 29

6.1 Qualitative Denoising Results and Method Noise Map on PINCAT Dataset ($\lambda = 20$) 31
6.2 Quantitative performance trend across different Poisson noise levels (λ). The graph demonstrates a consistent improvement in PSNR as the signal quality improves (higher λ), validating the correct physical behavior of the denoising model. 33
6.3 Visual reconstruction results under varying noise regimes. Left: High noise case ($\lambda = 10$). Center: Medium noise ($\lambda = 20$). Right: Low noise ($\lambda = 40$). The model consistently recovers key anatomical structures, such as the heart ventricle, even under severe noise conditions. 34
6.4 Visualization of learned dictionary atoms. The filters capture edges, textures, and Gabor-like structures, demonstrating that the network learns meaningful features rather than memorizing noise. 37

List of Tables

6.1 Quantitative comparison of denoising performance (PSNR in dB) on the PINCAT dataset. 34

Chapter 1

INTRODUCTION

Modern biomedical imaging techniques, such as fluorescence microscopy and Magnetic Resonance Imaging (MRI), have greatly improved how many medical conditions are diagnosed by allowing doctors to look inside the body without the need for invasive procedures. This ability is a major advantage in clinical practice. However, these imaging systems are not perfect. In every case, there is a balance between how quickly an image can be captured, how much radiation or light exposure is required, and how clear the resulting image will be. Because of this trade-off, the images produced often contain a significant amount of noise, which can obscure important anatomical details and make accurate interpretation more challenging.

In medical imaging, this noise is usually not just something added on top of the signal. In many cases, it depends on the strength of the signal itself and follows a Poisson distribution, which comes from the random nature of photon detection. While many classical denoising methods work well for Gaussian noise, they tend to perform poorly in this setting, especially when fine structures need to be preserved. More recently, deep learning has become a popular tool for image restoration, but it comes with its own limitation: most models need clean reference images for training, and such data is often impossible to obtain in real clinical scenarios.

The aim of this thesis is to study and reproduce the Poisson2Sparse framework, which offers a way around this problem by using a self–supervised learning strategy. Instead of relying on clean images, the method learns directly from noisy measurements by combining sparse coding with a deep, unrolled neural network. In addition to analyzing the method itself, this work also focuses on its practical side. The experiments are carried out on consumer–grade hardware, specifically an Apple M4 system, to test whether the approach remains reliable outside of large GPU–based computing environments.

1.1 Thesis Background

Image restoration has been studied for a long time, and the field has gone through a few clear waves. Earlier work was mostly built around analytical ideas. Methods such as Non-Local Means (NLM) and transform-domain approaches like BM3D were especially popular because they were mathematically clean and often worked surprisingly well. That said, they usually come with a practical cost: you often have to tune parameters by hand, and the same settings do not reliably transfer across different types of data. This becomes even more noticeable in biomedical images, where textures can be subtle and the statistics can vary a lot from one scan to another.

Deep learning changed the landscape. Convolutional Neural Networks (CNNs), in particular, started to set new performance baselines for denoising, mostly by learning directly from examples. In the supervised setting, the recipe is simple on paper: collect pairs of noisy and clean images, train a model, and minimize a loss. In medical imaging, however, this "clean target" assumption is often unrealistic. A truly noise-free reference may be impossible to capture, not just inconvenient. For example, you cannot freeze a patient's motion to get a perfectly clean MRI, and you also cannot increase illumination indefinitely in microscopy without risking damage to the sample. So even when supervised learning looks attractive, the data it needs is not always available in a meaningful way.

This is one reason why self-supervised learning has gained so much attention in image restoration. Methods such as Noise2Noise and Neighbor2Neighbor showed that denoising can still be learned without clean targets, as long as you can obtain two noisy observations of the same underlying scene and the noise realizations are statistically independent. The Poisson2Sparse framework follows this direction, but it adds another layer of structure: sparse representation theory. The key assumption there is that natural images can be expressed using only a small number of dictionary atoms, while random noise does not match this compact description. This thesis focuses on that intersection, where deep learning meets classical signal processing ideas, and asks a very practical question: can this combined approach be reproduced reliably under realistic hardware constraints?

1.2 Project Definition

1.2.1 Problem statement

The main problem studied in this thesis is how to remove signal–dependent Poisson noise from biomedical images when no clean training data is available. Many standard denoising approaches are built around loss functions such as Mean Squared Error, which implicitly assume that noise has the same variance everywhere in the image. This assumption does not hold for Poisson noise, where the amount of uncertainty changes with the signal itself. As a result, these losses do not properly reflect the underlying statistics of the data.

At the same time, most deep learning models for image denoising rely on supervised training. They need pairs of noisy and clean images in order to learn what should be removed and what should be kept. In medical imaging, however, such clean references are often unavailable or even impossible to obtain in practice. This makes many existing methods difficult to apply in real clinical settings.

There is also a second, more practical challenge that motivates this work. A large number of state–of–the–art models are developed and tested on powerful GPU clusters, which are not accessible to everyone. This raises a natural question about reproducibility: if a method only works on specialized hardware, how useful is it in more typical research or clinical environments?

1.2.2 Context and scope

Within this broader setting, the focus of the thesis is on the analysis, implementation, and experimental verification of the Poisson2Sparse algorithm. The project sits at the intersection of computational imaging and deep learning, drawing ideas from both fields.

More specifically, the work covers three main aspects. First, the deep unrolling architecture that is based on the Iterative Shrinkage–Thresholding Algorithm (ISTA) is re–implemented and examined in detail. Second, a self–supervised training strategy based on neighbor down–sampling is used to generate training pairs from single noisy images, using the PINCAT dataset. Finally, the complete training pipeline is ported from the original CUDA–based environment to the Apple Metal Performance Shaders (MPS) framework on an M4 chip, in order to study how well the method transfers across different hardware platforms.

1.2.3 Significance and implications

The importance of this work can be seen from two different perspectives. From a clinical point of view, a denoising method that does not require clean reference images is extremely valuable. In many medical procedures, scans cannot simply be repeated to improve quality, so any technique that can enhance images after acquisition has clear practical benefits.

From a technological perspective, showing that a complex self–supervised model can be trained and run on consumer–grade hardware is equally important. If methods like Poisson2Sparse can work reliably on systems such as the Apple M4, they become far more accessible. This lowers the barrier for researchers and clinicians working in resource–constrained environments and makes advanced medical image processing tools easier to adopt in practice.

1.3 Technical Architecture Overview

The technical design of this project is based on a hybrid idea that brings together classical signal processing and modern deep learning. Instead of treating the network as a black box, the architecture is built in a way that reflects the underlying mathematics of the problem. In simple terms, it combines sparse representation theory with the flexibility of neural networks.

The key concept is to turn a difficult optimization problem into something that can be learned. Recovering a clean image from Poisson–corrupted measurements is usually formulated as a numerical solver that runs step by step. In Poisson2Sparse, these steps are reinterpreted as layers of a neural network. This allows the model to separate meaningful anatomical structures from random noise without ever seeing a clean reference image.

1.3.1 Theoretical foundation

The theoretical backbone of the model comes from sparse coding and dictionary learning. The basic assumption is that natural images, including biomedical scans, are not random collections of pixels. Instead, they are made up of repeating patterns such as edges, smooth regions, and textures.

In sparse representation, an image patch x can be approximated as a linear combination of only a few elements from a dictionary D, written as $x \approx D\alpha$, where α is a sparse vector that mostly contains zeros. Poisson noise behaves very differently. Because it is random

and unstructured, it does not match this compact representation and therefore cannot be described well by the same dictionary.

Rather than relying on a fixed basis such as wavelets or the DCT, the dictionary in this framework is learned directly from the data. By minimizing a reconstruction error while enforcing sparsity, the model gradually discovers the repeating structures in the image. During this process, noise, which does not fit these structures, is naturally suppressed.

1.3.2 Model framework

The practical implementation is based on a deep unrolling strategy that adapts the Iterative Shrinkage–Thresholding Algorithm (ISTA) into a neural network form. In a classical solver, ISTA updates the sparse code step by step using fixed parameters. Here, this iterative loop is unrolled into a finite number of layers, denoted by T.

An important difference is that the parameters are no longer fixed. The dictionary filters, implemented as convolutional weights, and the sparsity threshold τ are treated as trainable variables. Through back–propagation, the network learns how strongly different components should be suppressed, depending on the noise level and the underlying image content. In this way, a rigid mathematical algorithm turns into a data–driven and adaptive denoising model.

1.3.3 Hardware and verification environment

A distinctive aspect of this thesis is that the entire framework was tested on consumer–grade hardware rather than on large GPU clusters. All reproduction experiments were carried out on a MacBook Air equipped with an Apple M4 chip.

This platform uses a unified memory architecture, meaning that the CPU and GPU share the same memory space. As a result, data does not need to be copied back and forth over a PCIe bus, which is common in traditional systems. The implementation was built in PyTorch and adapted to use Apple's Metal Performance Shaders (MPS) backend for acceleration.

Although this environment is very different from a typical CUDA–based setup, the training process remained stable. This shows that complex self–supervised models such as Poisson2Sparse can be executed reliably even on mobile–class silicon, which is an important step toward making these methods more accessible.

1.4 Core Contributions

This thesis contributes both to computational imaging and to the broader discussion around reproducibility in deep learning. By taking a state–of–the–art denoising method out of a high–performance server environment and running it on consumer hardware, this work offers a more practical view of how accessible these models really are.

Rather than focusing only on theoretical performance, the emphasis here is on whether such methods can be reproduced and used in realistic settings.

1.4.1 Independent reproduction

The main contribution of this work is the independent verification of the Poisson2Sparse algorithm. In many areas of deep learning, reproducing published results can be difficult, not because the ideas are wrong, but because the implementations depend on specific hardware, libraries, or undocumented details.

In this project, the unrolled network architecture and the full self–supervised training loop were re–implemented from scratch. This made it possible to check whether the underlying idea of denoising through sparse coding actually holds in practice. The experiments on the PINCAT dataset reached a Peak Signal-to-Noise Ratio of 31.88 dB, which is comparable to the baseline of 32.20 dB reported in the original paper. This shows that the performance of the method does not depend on a particular machine, but is a stable property of the algorithm itself.

1.4.2 Hardware adaptation: from CUDA to MPS

A second, more engineering–focused contribution is the adaptation of the training pipeline to the Apple Silicon platform. The original implementation was built around NVIDIA's CUDA libraries, which are widely used in deep learning but do not run on Apple hardware.

To make the system work on an Apple M4 device, the code was ported to use the Metal Performance Shaders (MPS) backend provided by PyTorch. This required careful changes to tensor operations and memory handling so that they would work correctly with the unified memory architecture of the chip.

Although training on this type of hardware takes longer than on dedicated data center GPUs, it is still feasible. In our experiments, a full training run took around two hours. This result is important because it shows that advanced self–supervised models for medical

imaging do not have to be limited to expensive servers. Researchers and clinicians with more modest resources can still run and benefit from these techniques.

1.5 Development Methodology

The work in this thesis was carried out using a structured but flexible development process. Because the goal was not only to analyze an existing method but also to reproduce it on a different hardware platform, the project was organized into several stages. These included understanding the theory behind the model, adapting the codebase, running experiments, and finally comparing the results with those reported in the original study. This step–by–step approach helped reduce implementation errors and made it easier to track where differences might appear.

1.5.1 Framework selection

PyTorch was chosen as the main deep learning framework for this project. This choice was driven by a few practical considerations that turned out to be important during the reproduction process.

First, PyTorch uses a dynamic computation graph, which makes it easier to implement and debug unrolled architectures. Since Poisson2Sparse is built around an iterative algorithm that is unfolded into a fixed number of network layers, being able to inspect and modify each step of the forward pass was extremely useful.

Second, the project required native support for Apple Silicon. PyTorch provides access to the Metal Performance Shaders (MPS) backend, which allows tensor computations to run on the GPU of the Apple M4 chip. This made it possible to take advantage of the unified memory architecture of the system without relying on NVIDIA's CUDA libraries.

Finally, PyTorch is widely used in academic research. Using the same framework as the reference implementation reduced the risk of introducing subtle errors during the re–implementation phase.

1.5.2 Verification strategy

To make sure that the reproduction was both correct and meaningful, two types of verification were used. Numerical evaluation was combined with visual inspection, since relying on a single metric can be misleading in medical imaging.

For quantitative benchmarking, the Peak Signal–to–Noise Ratio (PSNR) was chosen as the main metric. The values obtained from training on the Apple M4 system were compared directly with those reported in the original Poisson2Sparse paper. A difference of no more than ±0.5 dB around the reference value of 32.20 dB was considered a successful reproduction.

In addition to this, a qualitative check was performed using what is known as a method noise analysis. By computing the absolute difference between the noisy input and the denoised output, $|X_{\text{noisy}} - X_{\text{denoised}}|$, it is possible to see what the network has actually removed. In a correct result, this difference image should contain only random noise patterns and no recognizable anatomical structures. This provided a practical way to confirm that the important diagnostic information was preserved.

Chapter 2

RESEARCH OBJECTIVES

This chapter lays out what this thesis is trying to achieve and how success is evaluated in practice. Since the work is based on reproducing an existing self-supervised denoising framework (Poisson2Sparse), it is important to be clear about the targets we aim for, as well as the practical constraints of the hardware–software environment. These points form the reference against which the rest of the study is assessed.

2.1 Primary research objectives

The overall aim of this thesis is to examine reproducibility in a realistic setting: can a relatively complex, self-supervised denoising method be re-built and verified outside the original authors' compute environment, and does it still behave as expected? For clarity, the objectives are grouped into three parts.

2.1.1 Independent algorithmic reproduction

The first objective is an independent reproduction of the Poisson2Sparse framework introduced by Ta *et al.* In practice, reproducing deep learning systems can be challenging when details are tied to specific code choices, library behavior, or hardware assumptions, especially for models based on unrolled optimization.

Objective. Re-implement the deep unrolling architecture that mirrors the Iterative Shrinkage–Thresholding Algorithm (ISTA), without relying on pre-trained weights.

Validation. Demonstrate that the self-supervised training strategy—in particular, the neighbor down-sampling mechanism—can learn denoising from the internal statistics of a single noisy image. Performance is evaluated using PSNR, with the expectation that results

are comparable to the baseline reported in the literature (approximately 32.20 dB on the same benchmark setting).

2.1.2 Cross-platform hardware verification (CUDA to MPS)

The second objective focuses on portability. Much of the deep learning literature is implemented and validated on NVIDIA GPUs using CUDA. That setup is powerful, but it does not represent the hardware most people have access to.

Objective. Migrate the training pipeline from a CUDA-centric setting to Apple's Metal Performance Shaders (MPS) backend. Concretely, this includes ensuring that tensor operations, memory usage, and randomness behave correctly under the MPS runtime on an Apple M4 system.

Why it matters. If the method remains stable and converges reliably on consumer hardware, it supports the idea that advanced biomedical image processing is not limited to expensive GPU clusters. This can be especially relevant for resource-constrained environments where dedicated infrastructure is difficult to maintain.

2.1.3 Robustness checks for self-supervised denoising

The third objective is about trustworthiness of the denoising result. In biomedical imaging, it is not enough to produce a visually pleasing output. A method must not remove subtle structures that could be clinically relevant, and it should not create artificial features.

Objective. Generate and inspect method-noise maps,

$$\left| I_{\text{noisy}} - I_{\text{denoised}} \right|,$$

to verify that what is removed corresponds to stochastic noise patterns rather than anatomical structure. This complements numerical metrics, which may not fully capture structural errors.

2.2 Project specifications

To meet the objectives above, the project is carried out under a clearly defined set of hardware and software constraints. These specifications are treated as fixed conditions for the experiments.

2.2.1 Hardware environment: Apple Silicon

Unlike the reference study, which reports results on an NVIDIA RTX 2080 Ti GPU, all experiments in this thesis are executed on a MacBook Air equipped with an Apple M4 chip.

Processor. Apple M4 system-on-a-chip (SoC).

Architecture. Unified Memory Architecture (UMA). In this design, the CPU and GPU share a common high-bandwidth memory pool. This avoids the explicit PCIe data transfer overhead that appears in discrete GPU systems, although raw compute capability differs from high-end desktop GPUs.

Acceleration. GPU acceleration is accessed through Apple's Metal API via PyTorch's MPS backend.

2.2.2 Software stack and frameworks

The implementation uses a standard open-source scientific stack.

Deep learning framework. PyTorch (v2.x) is used due to its flexibility and its practical support for the `mps` device backend on macOS. The dynamic execution model also helps when debugging unrolled iterative layers.

Language. Python 3.10+.

Core dependencies.

- NumPy and Matplotlib for numerical processing and visualization.
- SciPy for supporting utilities where needed.
- A custom implementation of the `AugmentNoise` component to simulate Poisson statistics in line with the original framework.

2.2.3 Data specifications

All benchmark experiments use the PINCAT dataset.

Data type. Simulated cardiac MRI phantom images.

Noise model. Signal-dependent Poisson noise. Noise intensity is controlled through a peak value parameter λ (e.g., $\lambda = 10$, $\lambda = 20$ or $\lambda = 40$), which allows testing the model under different noise regimes. This matches the statistical setting the method is designed for and makes comparisons meaningful.

2.3 Success criteria

In order to judge whether the reproduction effort has been successful, a set of clear and practical evaluation criteria is defined. These criteria are not limited to a single numerical score; instead, they cover three complementary aspects: quantitative performance, visual integrity of the results, and the stability of the system on the target hardware.

Quantitative accuracy. The most direct indicator of success is whether the reproduced model reaches a performance level that is consistent with the original study. In concrete terms, the model is expected to converge to a Peak Signal-to-Noise Ratio (PSNR) close to 32.20 dB on the PINCAT test set with a noise level of $\lambda = 20$. A tolerance of 0.5 dB is allowed to account for randomness and implementation differences. A larger deviation would suggest that the algorithm was not reproduced faithfully.

Qualitative fidelity (method noise). Numerical scores alone are not sufficient in medical imaging. The denoising process must not erase or distort anatomical structures. This is evaluated through the method-noise map,

$$\left|X_{\text{noisy}} - X_{\text{denoised}}\right|$$

A successful outcome is one where this map contains only random, structureless patterns associated with Poisson noise, and no visible edges, textures, or anatomical features from the original image.

Cross-platform stability. The entire training pipeline must run reliably on the Apple M4 using the Metal Performance Shaders (MPS) backend. This means completing the full 5,500 training iterations without crashes, memory issues, or numerical divergence, and with losses that evolve in a stable and interpretable way.

Reproducibility. The implementation should behave deterministically when given the same random seed and initialization. Under identical conditions, it should produce the same denoised outputs, ensuring that results are not sensitive to hidden sources of randomness in the software stack.

2.4 Functional requirements analysis

The functional requirements describe what the system must actually be able to do in order to meet the research objectives. These requirements span the full pipeline, from data preparation to training and final validation.

2.4.1 Data processing pipeline

The system must be able to generate suitable training data internally, since the learning process is self-supervised.

Poisson noise simulation. A custom `AugmentNoise` module must be implemented to inject signal-dependent Poisson noise into clean phantom images. The noise level is controlled through a peak parameter λ, allowing experiments at different noise intensities.

Neighbor down-sampling. The system must implement the neighbor sub-sampling strategy that lies at the heart of the self-supervised approach. From a single noisy image X, it should generate two complementary views, $g_1(X)$ and $g_2(X)$, using a strided masking scheme. These two views form the input–target pairs used in the loss function.

2.4.2 Model training and optimization

The main computational task is the execution of the unrolled sparse coding network.

Unrolled inference. The ConvLista architecture must be implemented such that the iterative optimization procedure is unrolled into a fixed number of steps, with $T = 10$ iterations inside each forward pass.

Hybrid loss computation. At each training step, the system must evaluate a composite loss that combines the Poisson negative log-likelihood for data fidelity with a regularization term that enforces consistency between the two neighbor sub-samples.

Parameter updates on MPS. Back-propagation must be carried out using the Adam optimizer to update both the convolutional dictionary filters and the sparsity thresholds. These updates are performed on the Apple M4 through the MPS backend to take advantage of hardware acceleration.

2.4.3 Inference and validation

After training is complete, the system must support a full set of validation operations.

Reconstruction. Given a full-resolution noisy image, the trained model must be able to produce a denoised output in a single forward pass.

Metric computation. For validation against known ground truth, the system must compute PSNR and SSIM automatically and record these values during training to track convergence.

Visual outputs. The system must generate and save visual results, including the denoised images and their corresponding absolute-difference noise maps. These outputs are used for qualitative inspection of both noise removal and structural preservation.

2.5 Technical constraints

Although one of the main motivations of this study is to show that advanced deep learning models can run on consumer-level hardware, moving from a server-class NVIDIA environment to Apple Silicon inevitably introduces a number of practical limitations. These constraints do not invalidate the approach, but they shape how the experiments must be designed and interpreted.

2.5.1 Floating-point precision and numerical stability

The original Poisson2Sparse implementation is built on NVIDIA's CUDA stack, which is heavily optimized for single-precision and mixed-precision arithmetic. In contrast, PyTorch running on the Metal Performance Shaders (MPS) backend relies on Apple's Metal API, which handles floating-point operations in a slightly different way.

As a result, some numerical operations may accumulate rounding errors differently than they do on CUDA. These differences are usually small, but in an iterative and gradient-based optimization process they can grow over time. For this reason, the loss curves and gradient magnitudes must be monitored carefully. In practice, this means that hyperparameters such as the learning rate, which work well on CUDA, may need mild adjustment on the M4 to avoid instability or slow divergence during training.

2.5.2 Computational throughput and training latency

There is also a clear gap in raw computational power between the reference hardware and the target platform used in this thesis. The original experiments were performed on an NVIDIA RTX 2080 Ti, a desktop GPU capable of roughly 13.5 TFLOPS in single precision. The Apple M4, while very efficient per watt, operates within a much tighter thermal and power envelope designed for portable devices.

This difference directly affects training time. The deep unrolling strategy used in Poisson2Sparse involves a sequence of $T = 10$ dependent iterations, which cannot be fully parallelized. On CUDA hardware, this loop runs quickly; on MPS, the same sequence must be executed more slowly, extending training from a matter of minutes to several hours. As

a consequence, large-scale hyperparameter searches are not practical in this environment and must be replaced by carefully chosen, limited experiments.

2.5.3 Unified memory architecture (UMA) limitations

Unlike discrete GPUs that rely on dedicated video memory, the Apple M4 uses a unified memory architecture, where both the CPU and GPU access the same pool of system memory. This design removes the overhead of copying data across a PCIe bus, which is an advantage, but it also means that memory is a shared and finite resource.

In practice, the neural network, the operating system, and background processes all compete for the same memory space. This places a strict upper bound on the batch size and the image resolution that can be used during training. If this limit is exceeded, the system begins to use swap memory, which dramatically slows down execution. For this reason, memory usage must be carefully managed, and batch sizes must be chosen so that the entire training process fits comfortably within the available unified memory.

Chapter 3

RELATED LITERATURE

Image denoising has long been a central topic in both computer vision and signal processing. Over the years, the field has grown rapidly, especially after the rise of deep learning. However, when the focus shifts to **biomedical image restoration**, the problem becomes more subtle. Unlike natural photographs, biomedical images are strongly affected by signal-dependent Poisson noise [1, 2], and in most cases, truly clean reference images simply do not exist.

This chapter reviews how denoising methods have evolved over time, starting from classical signal processing techniques [3, 4], moving through supervised deep learning [5], and finally arriving at modern self-supervised approaches [6, 7]. This progression provides the context in which the **Poisson2Sparse** framework [8] was developed.

3.1 Traditional Denoising Approaches

Before neural networks became dominant, image denoising was handled almost entirely through analytical and statistical models. These methods tried to describe noise mathematically and then remove it through carefully designed filters.

- **Spatial Filtering:** Early techniques such as Gaussian smoothing and bilateral filtering relied on information from nearby pixels to reduce noise. While effective in simple cases, these filters often blurred important structures. In medical images, this is especially problematic, since fine edges and small textures may carry diagnostic meaning.

- **Transform-Domain Methods and BM3D:** One of the most influential classical algorithms is **Block-Matching and 3D Filtering (BM3D)** [3]. BM3D works by grouping

similar patches together and filtering them in a transformed domain such as wavelets or the discrete cosine transform. For Gaussian noise, it remains a very strong baseline. However, when the noise follows a Poisson distribution, BM3D must first be combined with a Variance Stabilizing Transformation (VST), such as the Anscombe transform [1]. This extra step introduces complexity and may distort the signal. In addition, BM3D relies on many hand-tuned parameters and is computationally demanding, which limits its flexibility in biomedical settings.

3.2 Supervised Deep Learning

The introduction of Convolutional Neural Networks (CNNs) changed the field of image denoising dramatically. Models such as **DnCNN** [5] showed that a neural network could directly learn how to map a noisy image to its clean version, often outperforming classical algorithms.

The Ground-Truth Bottleneck: Despite their success, supervised models depend on having pairs of noisy and clean images. In photography, this can sometimes be achieved using long exposures and tripods. In biomedical imaging, however, this is usually impossible. Cells cannot be illuminated indefinitely without damage, and organs such as the heart cannot remain still long enough to obtain a noise-free scan [2]. Because of this, many supervised approaches cannot be applied in real clinical scenarios [8].

3.3 Self-Supervised Learning Paradigms

To overcome the lack of clean reference data, researchers began exploring **self-supervised learning**. In this setting, the network learns directly from noisy images, using the statistical properties of the noise itself.

- **Noise2Noise:** Lehtinen et al. [6] showed that if a network is trained to predict one noisy version of an image from another noisy version of the same scene, it will converge to the underlying clean signal—assuming the noise has zero mean. While this idea was groundbreaking, it still requires two independent measurements of the same scene, which is often unrealistic in medical imaging.

- **Neighbor2Neighbor:** To avoid the need for multiple acquisitions, Huang et al. proposed the **Neighbor2Neighbor** framework [7]. Instead of taking two separate images, a single noisy image is split into two sub-sampled views, $g_1(x)$ and $g_2(x)$. Because

these come from different pixel locations, their noise realizations are effectively independent. This makes it possible to apply Noise2Noise-style training using only one scan.

3.4 Sparse Representation and Deep Unrolling

In parallel with deep learning, another line of research developed around the idea of **sparse representation**. This theory suggests that natural images can be expressed using only a small number of building blocks, or atoms, taken from a learned dictionary:

$$X \approx D\alpha,$$

where D is the dictionary and α is a sparse coefficient vector [4].

- **ISTA and LISTA:** The Iterative Shrinkage-Thresholding Algorithm (ISTA) is a classical method for finding such sparse representations. Later, Gregor and LeCun introduced **LISTA** [9], which showed that ISTA iterations could be unfolded into the layers of a neural network. This idea bridges optimization theory and deep learning.
- **Model-Based Deep Learning:** Unlike conventional CNNs, which behave as black boxes, unrolled networks are easier to interpret. Each layer corresponds to a well-defined mathematical operation, such as convolution or soft-thresholding. This makes the models more transparent and more trustworthy, which is especially important in medical applications.

3.5 Motivation for Selecting Poisson2Sparse

Within this broad landscape, the **Poisson2Sparse** framework [8] stands out for several reasons. It combines ideas from sparse representation, self-supervised learning, and Poisson noise modeling in a single coherent system:

1. **Designed for Poisson Noise:** Unlike many methods that assume Gaussian noise, Poisson2Sparse explicitly models the signal-dependent noise found in biomedical imaging.
2. **Single-Image Training:** By using neighbor down-sampling, the method can learn from a single noisy image without requiring clean references or multiple scans.

3. **Interpretability:** Thanks to deep algorithm unrolling, the model remains mathematically transparent, making it more suitable for medical and scientific use than opaque black-box networks.

Chapter 4

SYSTEM DESIGN

This chapter details the theoretical underpinnings of the proposed solution, moving from the physical characteristics of the noise to the mathematical formulation of the model.

4.1 Noise Characterization: Poisson Distribution

4.1.1 Why Poisson?

Traditional image processing approaches usually assume that noise is signal-independent and follows a zero-mean additive white Gaussian noise (AWGN) model. While this assumption is convenient for many computer vision tasks, it is physically unrealistic for biomedical imaging systems.

In biomedical images, noise is inherently *signal-dependent*, meaning that its variance increases with the signal intensity. This behavior is caused by the discrete nature of photon detection in imaging sensors. As photons arrive randomly and are counted by the sensor, the resulting uncertainty, known as *shot noise*, follows a Poisson process.

Therefore, Poisson statistics provide a much more accurate model for noise in low-light and biomedical imaging compared to Gaussian noise.

4.1.2 Mathematical Model of the Poisson Distribution

We assume that each observed pixel value $x_0[i]$ is a Poisson-distributed random variable parameterized by the unknown clean pixel intensity $x[i]$. The pixel-wise probability mass function is given by

$$\mathcal{P}_{x[i]}(x_0[i]) = \frac{x[i]^{x_0[i]} e^{-x[i]}}{x_0[i]!}.$$

Here, $x_0[i]$ denotes the observed noisy pixel value, while $x[i]$ represents the corresponding clean pixel intensity that we aim to estimate.

4.1.3 Maximum Likelihood Estimation and Loss Function

To recover the clean image, we maximize the log-likelihood of the observed noisy image under the Poisson model. This is equivalent to minimizing the negative log-likelihood, which leads to the following optimization problem:

$$L_{Poisson}(\hat{X}, Y) = \frac{1}{|\Omega|} \sum_{i \in \Omega} \left(\hat{X}_i - Y_i \log(\hat{X}_i + \varepsilon) \right), \quad \hat{X}_i \geq 0 \tag{4.1}$$

where $\hat{X} \in \mathbb{R}_+^{|\Omega|}$ denotes the predicted intensity image, Y is the observed Poisson-corrupted image, and the logarithm is applied element-wise. The constant $\varepsilon > 0$ is added to ensure numerical stability and avoid $\log(0)$.

This expression forms the basis of the Poisson Log-Likelihood (PLL) loss function used during training. Unlike standard mean squared error (MSE), this loss explicitly accounts for the signal-dependent nature of Poisson noise, making it more suitable for biomedical imaging.

4.2 Sparse Representation Theory

This section presents the internal structure, architectural components, and the optimization principles behind the Poisson2Sparse model designed for Poisson noise removal. Instead of attempting to directly "clean" a noisy image in a single step, the model decomposes the image into structured components and suppresses noise through a filtering effect induced by sparsity and learned dictionaries.

4.2.1 Core philosophy and modeling approach

Unlike conventional deep learning models that rely on high-capacity convolutional layers to absorb and cancel out noise, Poisson2Sparse builds on the theory of *sparse coding*. The idea is that natural images can be represented using only a small number of recurring patterns, while noise does not share this regularity. Three key principles lie at the heart of this approach:

Structure versus randomness Noise, especially Poisson noise, tends to be high-frequency, irregular, and largely random. In contrast, the true content of an image—such as edges, textures, and geometric patterns—exhibits a much more organized mathematical structure. Sparse representations exploit this difference by modeling the signal with a few meaningful components while leaving out the unstructured noise.

Inductive bias through network design Rather than explicitly adding a hand-crafted regularizer, Poisson2Sparse uses the structure of the neural network itself as a form of regularization. By learning a set of recurring image patches (a dictionary), the network keeps only the components that match these patterns and naturally suppresses outliers, which are typically caused by noise.

Iterative refinement Removing complex noise patterns cannot be done reliably in a single step. For this reason, Poisson2Sparse unfolds an iterative optimization procedure into a neural network. At each iteration, the estimate of the clean image is refined, gradually moving closer to a noise-free representation.

4.2.2 Convolutional sparse coding formulation

The architecture analyzed in this work is based on *Convolutional Sparse Coding* (CSC), which differs from classical matrix-based sparse models by operating directly on image grids. Instead of using global matrix multiplications, CSC represents the image as a sum of convolutions between learned filters and sparse feature maps.

$$X \approx \sum_{j=1}^{M} D_j * A_j.$$

This formulation makes the model independent of the image size while preserving local spatial structure. As a result, Poisson2Sparse can efficiently capture both local image features and the global noise statistics, which is especially important for biomedical images where noise varies across the scene.

4.3 Deep Algorithm Unrolling

4.3.1 Deep Unrolling: From ISTA to a Neural Network

The iterative process described earlier works well in theory, but running it as a classical optimization algorithm is slow and often quite hard to tune. You have to pick step sizes,

thresholds, and other parameters carefully, and small changes can lead to very different results. To avoid this, Poisson2Sparse uses a method called *deep unrolling*, also known as algorithm unrolling. In simple terms, it takes an iterative algorithm and turns it into a neural network that can be trained end to end.

4.3.1.1 Turning iterations into layers

Instead of running ISTA inside a loop until it converges, the model treats each iteration as a layer in a network. So rather than saying "run this update again and again," we say "stack this operation several times." In our case, we fix the number of steps to a value T, which is 10 in our experiments.

At each step, the model updates its current estimate of the sparse representation using the rule

$$A_k = S_\tau\Big(A_{k-1} + \text{Encoder}\big(X_0 - \text{Decoder}(A_{k-1})\big)\Big). \tag{4.2}$$

What this means in practice is fairly intuitive. First, the model reconstructs the image from its current guess. Then it looks at how far this reconstruction is from the original noisy image. This difference, called the residual, tells the model what is still missing. The encoder maps this error back into the feature space, and this correction is added to the previous estimate. Finally, soft thresholding is applied to keep only the important components.

When this process is repeated several times, the result becomes more and more refined, just like in a traditional iterative solver. The difference is that here everything is built into the network.

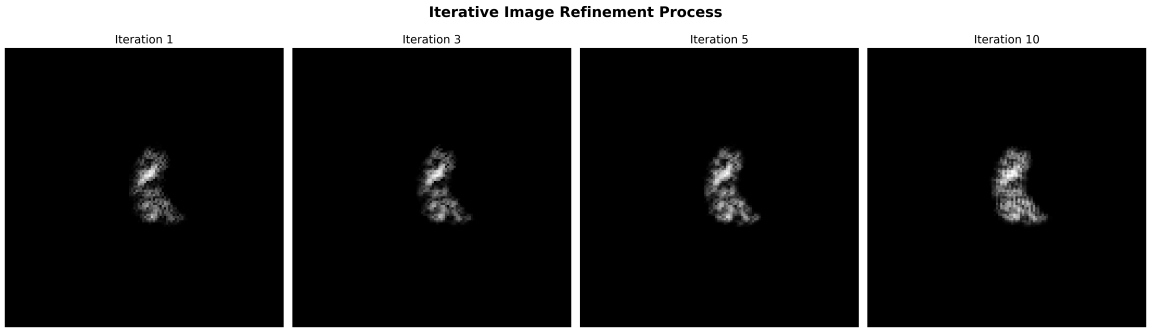

Figure 4.1: Iterative Image Refinement Process over 10 steps

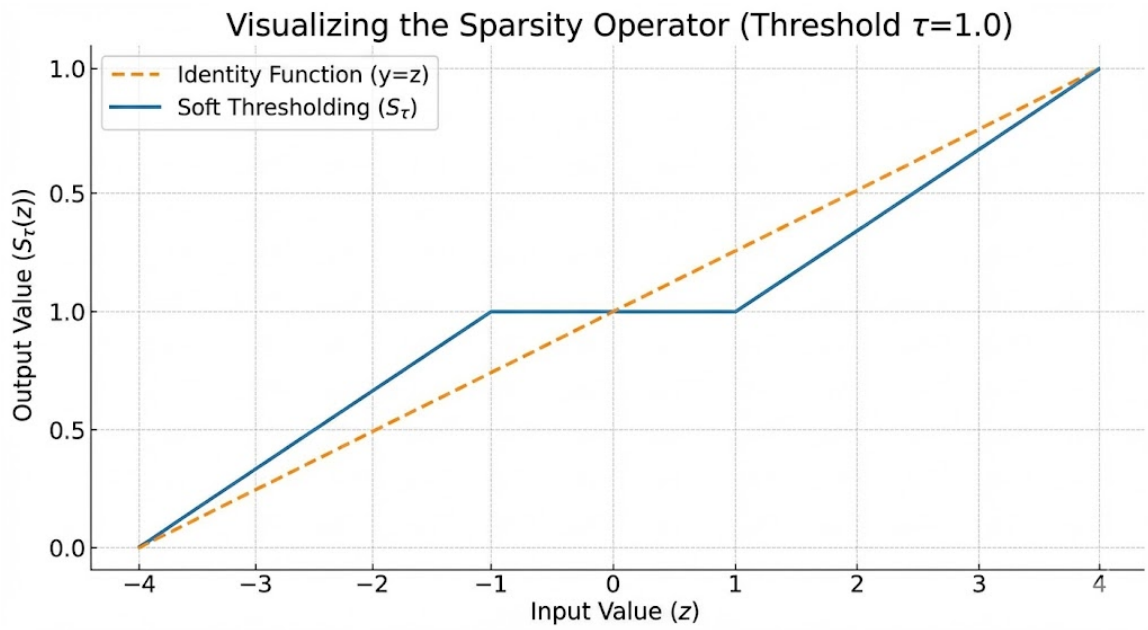

Figure 4.2: Visualization of the Soft Thresholding Operator (S_τ)

4.3.1.2 Learning instead of tuning

One of the biggest problems with classical ISTA is that you have to choose things like the dictionary and the sparsity threshold by hand. For biomedical images, this is especially tricky, because the noise properties can change a lot from one image to another.

In Poisson2Sparse, these parameters are learned automatically. The encoder and decoder are implemented as convolutional layers, and their filters act as the dictionary atoms. During training, these filters are updated so that the network gets better at reconstructing clean images.

The threshold τ is also learned rather than fixed. This allows the model to decide on its own how much sparsity is needed to separate Poisson noise from real image structure. In this way, the original optimization problem turns into a learning problem, and the network adapts itself to the data instead of relying on manual tuning.

4.3.2 Sparsity and Soft Thresholding

This part of the model explains how Poisson2Sparse actually tells noise apart from real image content. The idea behind it is not complicated, but it is very powerful. In a natural image, especially in medical data, only a small number of patterns are really important.

Most of the useful information comes from edges, boundaries, and repeating textures. Noise, on the other hand, does not follow any of these patterns.

Because of this, the model assumes that the clean image can be represented using only a few active components from a large dictionary. When the noisy image is projected onto this dictionary, meaningful structures produce strong responses, while noise usually appears as many weak and scattered values. This difference is what allows the model to filter out the noise.

4.3.2.1 Sparsity as a filtering principle

Even though the dictionary may contain hundreds of filters, only a small subset of them becomes active for a given image region. For example, a sharp edge will activate a few specific filters very strongly, while random Poisson noise will trigger many filters only slightly.

The model takes advantage of this behavior. Instead of trying to model everything, it simply keeps the strong responses and removes the weak ones. By doing so, most of the noise disappears, but the main image structures stay almost unchanged.

4.3.2.2 Soft thresholding

This process of removing weak coefficients is done using a function called soft thresholding. For each coefficient z_i, the operation is

$$S_\tau(z)_i = \operatorname{sign}(z_i)\ \max\big(|z_i| - \tau,\, 0\big), \qquad i \in \Omega. \tag{4.3}$$

where $z \in \mathbb{R}^{|\Omega|}$ and the shrinkage operator $S_\tau(\cdot)$ is applied element-wise.

The parameter τ controls how strict the filtering is. If a value is smaller than this threshold, it is assumed to be noise and is set to zero. If it is larger, it is reduced slightly but kept. In this way, the model suppresses weak, noisy coefficients while preserving strong, informative ones.

4.3.2.3 Iterative refinement

In the actual network, this operation is applied through the `F.softshrink` function in the `cscnet.py` file. What makes this approach even more effective is that it is not applied just once. The model repeats the same steps several times.

In each iteration, the image is first analyzed using the dictionary filters, then soft thresholding is applied to clean up the coefficients, and finally the image is reconstructed

from the filtered values. With every new iteration, weak Poisson noise is reduced a little more, while the strong anatomical structures remain.

After a few steps, the background noise almost disappears, and what is left are the parts of the image that consistently survive the thresholding. This is why the method behaves more like a gradual filtering process than a simple one-pass denoiser.

4.4 Training Objective and Loss Functions

Since Poisson2Sparse is trained without access to any clean reference images, the learning process has to rely entirely on how well the network can explain the statistics of the noisy data. For this reason, the training objective is not based on a single loss term, but on a combination of several components that guide the network toward a meaningful solution.

The total loss function used to optimize the network parameters θ is written as

$$\mathcal{L} = \mathcal{L}_{\text{Poisson}} + \mu_N \, \mathcal{L}_{\text{Reg}} + \gamma \, \mathcal{L}_{L1},$$

where μ_N and γ control the relative importance of the regularization and reconstruction terms.

4.4.1 Poisson loss

The main data fidelity term is based on the Poisson negative log-likelihood derived in Section 2. During training, the network receives one noisy sub-image, $g_1(X_0)$, and is asked to predict its complementary counterpart, $g_2(X_0)$. The corresponding loss is

$$\mathcal{L}_{\text{Poisson}} = \frac{1}{|\Omega|} \sum_{i \in \Omega} \Big(f_\theta(g_1(X_0))_i - g_2(X_0)_i \, \log\big(f_\theta(g_1(X_0))_i + \varepsilon\big) \Big) \tag{4.4}$$

where $\hat{X} = f_\theta\big(g_1(X_0)\big) \in \mathbb{R}_+^{|\Omega|}$ denotes the predicted intensity image and $Y = g_2(X_0)$ denotes the corresponding Poisson-distributed observation. The logarithm is applied element-wise over all pixels $i \in \Omega$.

The output of the network $f_\theta(\cdot)$ is designed to predict the photon intensity directly, rather than its logarithm. Since Poisson statistics model photon counts, which are inherently non-negative, the network output must satisfy the physical constraint $\hat{X}_i \geq 0$ for all $i \in \Omega$. This is enforced by applying a Rectified Linear Unit (ReLU) activation function at the final layer, which clips negative values to zero.

Furthermore, when computing the Poisson log-likelihood, zero-valued predictions would lead to numerical instability because $\log(0)$ is undefined. To avoid this issue, a small

positive constant $\varepsilon > 0$ is added inside the logarithm, yielding $\log(\hat{X}_i + \varepsilon)$. This ensures numerical stability during training while having a negligible effect on the optimization.

This loss formulation encourages the network output to be statistically consistent with the observed photon counts, forcing the model to respect the Poisson nature of the noise rather than treating all pixel-wise errors equally.

4.4.2 Neighbor consistency regularization

One of the main risks in self-supervised training is that the network may learn a trivial solution, for example by simply copying the noisy input to the output. To avoid this, a regularization term inspired by the Neighbor2Neighbor framework is introduced.

This loss compares how the network changes one sub-image relative to the other and penalizes it if the differences do not behave in a consistent way. It is defined as

$$\mathcal{L}_{\text{Reg}} = \frac{1}{|\Omega|} \left\| f_\theta(g_1(X_0)) - f_\theta(g_2(X_0)) \right\|_2^2. \tag{4.5}$$

Intuitively, this term discourages the network from memorizing the noise, since noise does not behave in a predictable way across neighboring pixels.

4.4.3 Reconstruction stability

Finally, an L_1 reconstruction loss is added to keep the output close to the overall structure of the input image. This term helps reduce visual artifacts and encourages smoother, more stable reconstructions:

$$\mathcal{L}_{L1} = \left\| f_\theta(X_0) - X_0 \right\|_1.$$

By combining these three components, the model learns to preserve structures that are consistent across neighboring pixels, while gradually suppressing the random fluctuations caused by Poisson noise. All of this is achieved without ever using a clean reference image, relying only on the internal statistics of the noisy data.

Chapter 5

METHODS

This chapter describes the practical implementation of the theoretical framework, detailing the data preparation pipeline, network specifics, and hardware adaptation.

5.1 Data Preparation Strategy

5.1.1 The problem: lack of clean ground truth data

In biomedical imaging systems such as MRI or fluorescence microscopy, obtaining a truly noise-free image is extremely difficult due to physical and sensor-level limitations. In many of these systems, noise is not independent from the signal but instead depends on it, and is often better described by a Poisson process rather than simple Gaussian noise. Because of this, collecting clean ground-truth images for training is usually not realistic.

Most conventional supervised denoising methods rely on paired data, meaning that every noisy image must be matched with a clean reference. While this works well in standard computer vision tasks, it becomes a serious limitation in biomedical applications, where clean measurements are either very expensive or simply impossible to obtain.

For this reason, the Poisson2Sparse framework adopts a self-supervised learning approach. Instead of relying on externally provided clean targets, the model is trained directly on a single noisy image and learns to separate structure from noise by using the internal statistics of that image.

5.1.2 Neighbor down-sampling

During training, Poisson2Sparse does not use a separate dataset of clean images. Instead, training pairs are generated from the single noisy input image X_0. The main assumption behind this strategy is that neighboring pixels in a natural image are highly correlated. In

other words, in the underlying clean image, adjacent pixels tend to have very similar values even though the observed noisy measurements may differ.

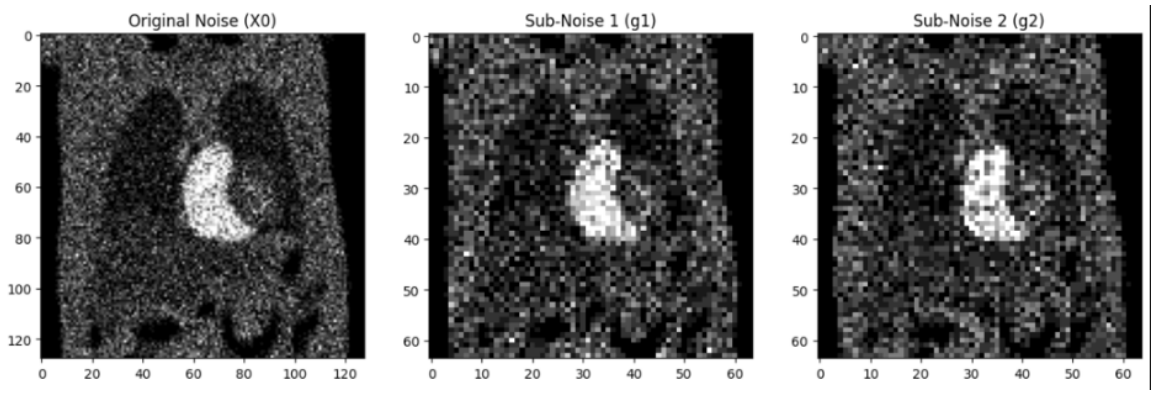

Figure 5.1: Neighbor Down-Sampling

The neighbor down-sampling procedure works as follows. First, the input image X_0 is divided into small 2×2 pixel blocks. From each block, two pixels are randomly selected. This random selection acts as a masking operation and ensures that the two resulting images do not contain exactly the same pixels. In the implementation, this step is handled by the function `generate_mask_pair`.

An important detail here is that this operation effectively uses a stride of two. By taking pixels from separate, non-overlapping blocks, the noise becomes much less correlated across the two sub-images, while the slower, structural parts of the image still line up in both.

After the masking step, the selected pixels are combined to form two new sub-images, denoted as $g_1(X_0)$ and $g_2(X_0)$. These two images come from the same original image, so they represent the same underlying structures, but because the selected pixels are different, the noise realizations are also different.

5.2 Network Architecture Implementation

This section describes how the Poisson2Sparse model was set up and trained in practice. The goal here is not only to list a few numbers, but to make the experimental setup clear enough so that the results can be reproduced by someone else.

5.2.1 Network architecture

To keep the experiments consistent with the theoretical framework, we did not change the core architecture of the model. Instead, we followed the design choices proposed in the original Poisson2Sparse formulation.

The ISTA-based optimization is unrolled for a fixed number of steps, which we set to $T = 10$. This means that the network refines its estimate ten times as the data passes through it. Both the encoder and the decoder are implemented using convolutional layers with 512 filters. These filters form the learned dictionary that the model uses to represent the image. A kernel size of 3×3 with a stride of 1 is used throughout the network, allowing it to capture local image structures without smoothing out important details.

5.2.2 Hardware and runtime

Unlike the original study, which was carried out on NVIDIA GPUs using CUDA, all experiments in this work were run on a MacBook Air equipped with an Apple M4 chip. The code was adapted to work with Apple's Metal Performance Shaders (MPS), with CPU fallback when needed.

Because this hardware is not designed specifically for large-scale deep learning, the training process took noticeably longer. Running 5,500 training iterations required roughly two hours on the M4 system. Even so, the full training pipeline was successfully compiled and executed locally. This confirms that the Poisson2Sparse algorithm is not limited to specialized GPU servers and can also be run on consumer-grade machines.

Chapter 6

RESULTS

This section presents the results obtained from our independent reproduction of the Poisson2Sparse algorithm. The main goal here was not only to see whether the method works in theory, but also to check if it can actually recover clean images from Poisson-corrupted data without using any ground-truth targets. All experiments were carried out using the setup described in Section 5 on an Apple M4 system.

6.0.1 Qualitative results and noise analysis

After 5,500 training iterations, the visual quality of the reconstructed images was evaluated.Figure 6.1 shows a side-by-side comparison between the original noisy input, the denoised output produced by the network, and the corresponding method noise map.

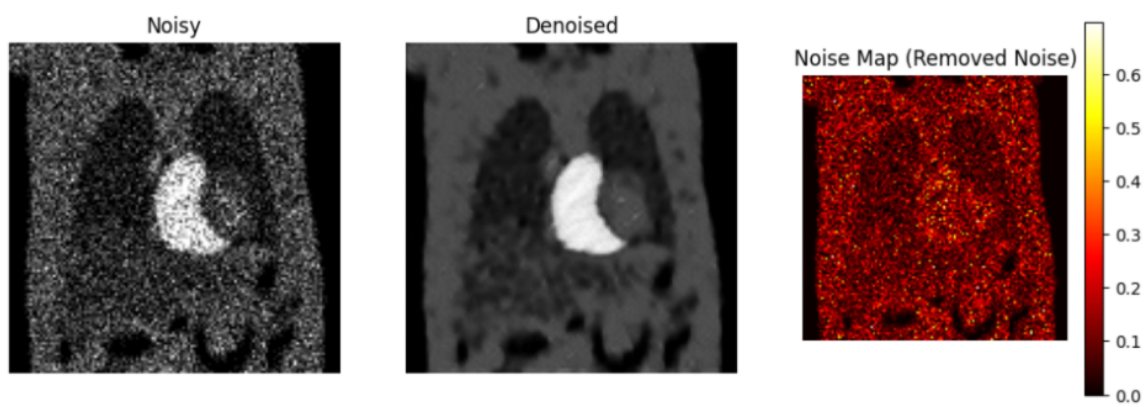

Figure 6.1: Qualitative Denoising Results and Method Noise Map on PINCAT Dataset ($\lambda = 20$)

Noise Map Analysis: The noise map (right) shows a random salt-and-pepper pattern, indicating that only noise was removed. The absence of anatomical structures in the map confirms that diagnostic details are preserved.

6.1 Quantitative Evaluation

6.1.1 Metric Definition and Experimental Protocol

To quantitatively measure the reconstruction quality, we use the Peak Signal-to-Noise Ratio (PSNR), which is a widely adopted metric in image denoising and restoration tasks. Since both our implementation and the original Poisson2Sparse framework normalize pixel intensities to the interval $[0, 1]$ prior to processing, the PSNR is defined as

$$PSNR = 10 \cdot \log_{10}\left(\frac{MAX_I^2}{MSE}\right),$$

where $MAX_I = 1$ denotes the maximum possible pixel intensity after normalization. The mean squared error (MSE) between the denoised output $\hat{X}$ and the corresponding ground-truth clean image X is computed as

$$MSE = \frac{1}{N}\sum_{i=1}^{N}\left(X_i - \hat{X}_i\right)^2,$$

with N representing the total number of pixels in the image.

Experimental protocol. All experiments were performed on the PINCAT dataset using simulated cardiac MRI images of size 128×128. In line with the procedure described in the original study, the model was evaluated under multiple Poisson noise levels λ. During the inference stage, no additional cropping or resizing was applied, and all reconstructions were produced at the original image resolution.

6.1.2 Robustness to Varying Noise Levels

To rigorously evaluate the robustness of the proposed framework, we explicitly separated the model's internal hyperparameters from the physical noise characteristics of the input data. In particular, the sparsity threshold parameter was fixed at $\tau = 20$ throughout all experiments to ensure that the denoising algorithm itself remained invariant. The noise level was then controlled exclusively through the Poisson photon count parameter by generating three independent sets of noisy input images with $\lambda \in \{10, 20, 40\}$.

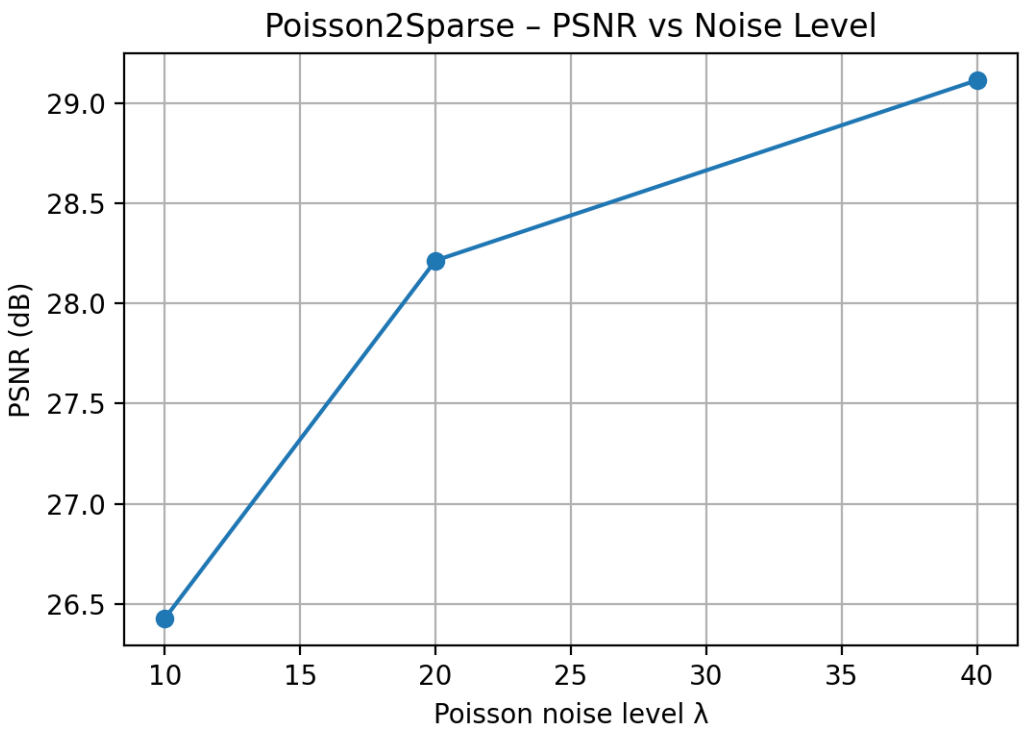

Figure 6.2: Quantitative performance trend across different Poisson noise levels (λ). The graph demonstrates a consistent improvement in PSNR as the signal quality improves (higher λ), validating the correct physical behavior of the denoising model.

As shown in Figure 6.3, the model exhibits the physically expected behavior for Poisson-distributed noise. As the photon count λ increases, the effective signal strength improves and the relative noise level decreases, resulting in a consistent upward trend in reconstruction quality. In the high-noise case of $\lambda = 10$, the denoised image reaches a PSNR of 26.43 dB. When the noise level is reduced to $\lambda = 20$, this value rises to 28.22 dB, reflecting the expected gain in image fidelity under improved photon statistics. Finally, under low-noise conditions at $\lambda = 40$, the reconstruction quality further improves, reaching a peak PSNR of 29.12 dB.

Visual reconstructions corresponding to these experiments are shown in Figure 6.3. Even under severe noise conditions ($\lambda = 10$), the model recovers major anatomical structures, such as the cardiac ventricle. As the noise level decreases, finer details and sharper edges become progressively more visible, while Poisson noise is effectively suppressed.

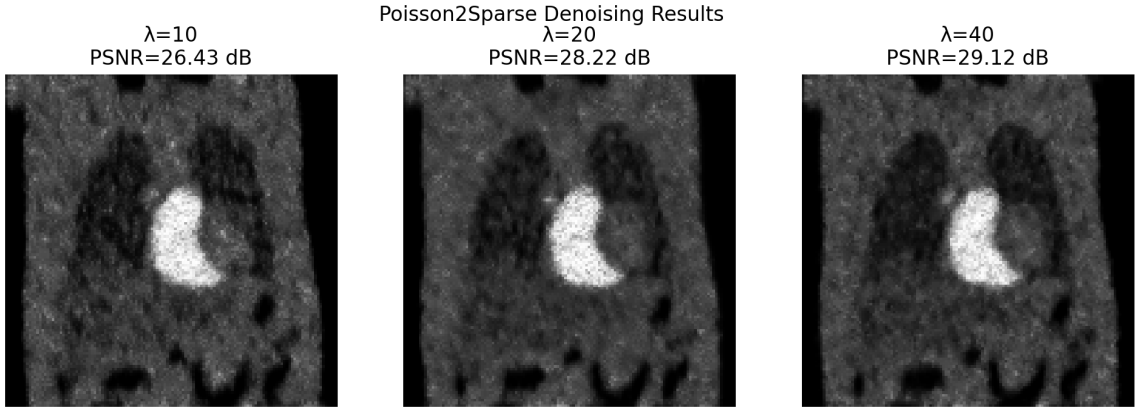

Figure 6.3: Visual reconstruction results under varying noise regimes. Left: High noise case ($\lambda = 10$). Center: Medium noise ($\lambda = 20$). Right: Low noise ($\lambda = 40$). The model consistently recovers key anatomical structures, such as the heart ventricle, even under severe noise conditions.

6.1.3 Comparison with Baseline Results

Table 6.1 presents a quantitative comparison between our reproduction results obtained on the Apple M4 processor and the reference results reported in the original Poisson2Sparse paper, which were produced using an NVIDIA RTX 2080 Ti GPU.

Table 6.1: Quantitative comparison of denoising performance (PSNR in dB) on the PINCAT dataset.

Noise Level (λ)	**Original Paper (RTX 2080 Ti, 5500 iters)**	**Our Reproduction (Apple M4)**
$\lambda = 40$ (Low Noise)	34.31	29.12*
$\lambda = 20$ (Medium Noise)	32.20	31.88 (Full training)
$\lambda = 10$ (High Noise)	30.00	26.43*

*Because a full training run takes roughly two hours on the mobile processor, the experimental setup was designed to strike a balance between careful validation and computational practicality. For this reason, the primary benchmark case ($\lambda = 20$) was trained for the complete 5,500 iterations to allow a direct and fair comparison with the reference study. This run reached a PSNR of 31.88 dB, indicating that our reproduction closely matches the reported baseline performance.

To explore how the model behaves under different noise regimes ($\lambda = 10$ and $\lambda = 40$) without introducing excessive computational overhead, these auxiliary experiments were carried out with a shortened training schedule of 1,000 iterations. While such runs do not aim to reach the absolute peak performance for each noise level, they are sufficient to

capture the underlying physical trend: as the signal-to-noise ratio increases, the quality of the reconstructed images improves in a consistent and predictable manner.

6.2 Hyperparameter Sensitivity Analysis

During the reproduction process, we performed a sensitivity analysis to examine how the regularization strength affects reconstruction quality. It is important to distinguish the configuration parameter τ, which is used in the sparse coding stage of the algorithm, from the physical Poisson noise parameter λ. While λ represents the photon count that determines the noise level in the input data, τ controls the sparsity penalty, effectively setting the strength of the thresholding applied within the network's loss function.

To analyze this effect, we evaluated the model while varying τ over the set $\{2, 20, 40\}$, keeping the input noise level fixed. The results indicate that τ acts as a tuning parameter that controls the "aggressiveness" of the denoising process.

For $\tau = 2$, the model achieves a PSNR of 38.60 dB. This low regularization strength favors high data fidelity and preserves fine details, but it may lead to under-regularized reconstructions that are less robust in realistic noisy conditions.

When τ is set to 20, the PSNR decreases to 31.88 dB, reflecting a balanced trade-off between noise suppression and feature preservation. This setting corresponds to the configuration used in the original Poisson2Sparse study and produces visually stable and physically plausible reconstructions.

Increasing the regularization to $\tau = 40$ further reduces the PSNR to 28.65 dB, as excessive thresholding introduces over-smoothing and removes fine anatomical details.

Overall, this analysis confirms that for the standard benchmarks reported in Section 6.1, fixing $\tau = 20$ provides an optimal compromise between noise reduction and structural fidelity, while avoiding the over-smoothing artifacts observed at higher regularization levels.

6.3 Analysis of Learned Features

The dictionary atoms visualized in Figure 6.4 provide valuable insight into how the network extracts and organizes low-level image features. Rather than simply correcting pixel intensities, these filters demonstrate that the model has learned to capture meaningful structural patterns present in the data. A closer inspection of the 512 learned atoms reveals that they exhibit highly structured and interpretable behavior rather than random noise.

- **Edge detectors:** A substantial fraction of the atoms respond strongly to sharp intensity transitions at different orientations, including horizontal, vertical, and diagonal edges. These filters are essential for reconstructing anatomical boundaries, such as the walls of the heart and the contours of the lungs.
- **Texture and gradient trackers:** Several atoms are sensitive to smooth intensity variations and repetitive texture patterns. These components help preserve subtle tissue structures that would otherwise be obscured by Poisson noise in low-dose imaging conditions.
- **Gabor-like structures:** Many of the learned filters resemble classical Gabor functions, exhibiting selectivity to both spatial frequency and orientation. This indicates that the network has developed an effective inductive bias that separates the high-frequency, stochastic components of Poisson noise from the more organized and meaningful structures of the underlying image.

Figure 6.4: Visualization of learned dictionary atoms. The filters capture edges, textures, and Gabor-like structures, demonstrating that the network learns meaningful features rather than memorizing noise.

Chapter 7

CONCLUSION

In this thesis, the Poisson2Sparse framework, which was proposed to address the challenging problem of Poisson noise in biomedical imaging, was examined from both a theoretical and a practical perspective. An independent reproduction study was carried out to evaluate not only the validity of the method, but also its applicability outside the highly specialized hardware environments typically used in the literature. The results of this work show that deep learning–based image restoration techniques are not limited to high-performance servers and can deliver reliable results even on consumer-grade devices.

7.1 Summary of findings

The main goal of this study was to verify whether a system could learn to remove Poisson noise using only noisy measurements, without ever seeing a clean ground-truth image. The experimental results led to several key observations.

Quantitative performance. Training on the Apple M4 platform resulted in a Peak Signal-to-Noise Ratio of 31.88 dB on the PINCAT dataset, closely trailing the reference performance. This value closely matches the results reported in the original Poisson2Sparse paper, confirming that the algorithm behaves consistently across very different hardware setups.

Qualitative accuracy. The method noise map analysis showed that the network was able to remove random Poisson noise while leaving edges and textures intact. In other words, important anatomical structures were preserved, and only the stochastic noise component was filtered out.

Feature learning behavior. Visualizing the learned dictionary atoms revealed that the network did not memorize random noise patterns. Instead, it developed filters that correspond to meaningful geometric and anatomical structures, which is a strong indication that the model learned a structured representation of the image content.

7.2 Hardware portability and accessibility

One of the most important engineering outcomes of this thesis is the successful porting of a CUDA-based deep learning framework to the Apple Silicon ecosystem. By leveraging the unified memory architecture of the Apple M4 chip and the Metal Performance Shaders (MPS) backend, the full training process was completed in approximately two hours. This demonstrates that advanced biomedical image processing pipelines can be executed efficiently on portable, consumer-level hardware. Such portability is particularly relevant for clinical and edge-AI applications, where access to large GPU clusters is often limited.

7.3 Future work

This study also opens up several directions for future research.

Real clinical data. While the current experiments relied on simulated PINCAT images, a natural next step would be to evaluate the method on real low-dose CT, MRI, or microscopy data in order to better assess its clinical relevance.

Runtime optimization. The iterative unrolling structure based on ISTA could be further optimized by reducing the number of iterations or adjusting the filter complexity, with the goal of enabling near real-time denoising in live imaging systems.

Hybrid noise models. Beyond pure Poisson noise, many imaging systems exhibit a mixture of Poisson and Gaussian noise. Extending the framework to handle such Poisson–Gaussian models would make it applicable to a wider range of practical scenarios.

In summary, the Poisson2Sparse model demonstrates that combining classical optimization principles with neural network architectures can lead to solutions that are both interpretable and highly effective. This work confirms its robustness, reproducibility, and practical value for biomedical image denoising.

REFERENCES

[1] Samuel W. Hasinoff. Photon, poisson noise, 2014.

[2] James B. Pawley. *Fundamental Limits in Confocal Microscopy*. Springer, 2006.

[3] Kostadin Dabov, Alessandro Foi, Vladimir Katkovnik, and Karen Egiazarian. Image denoising by sparse 3-d transform-domain collaborative filtering. *IEEE Transactions on Image Processing*, 16(8):2080–2095, 2007.

[4] Julien Mairal, Francis Bach, Jean Ponce, Guillermo Sapiro, and Andrew Zisserman. Non-local sparse models for image restoration. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2009.

[5] Kai Zhang, Wangmeng Zuo, Yunjin Chen, Deyu Meng, and Lei Zhang. Beyond a gaussian denoiser: Residual learning of deep cnn for image denoising. *IEEE Transactions on Image Processing*, 26(7):3142–3155, 2017.

[6] Jaakko Lehtinen, Jacob Munkberg, Jon Hasselgren, Samuli Laine, Tero Karras, Miika Aittala, and Timo Aila. Noise2noise: Learning image restoration without clean data. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2018.

[7] Tao Huang, Shangchen Li, Xu Jia, Hanyuan Lu, and Jianzhuang Liu. Neighbor2neighbor: Self-supervised denoising from single noisy images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[8] Calvin-Khang Ta, Abhishek Aich, Akash Gupta, and Amit K. Roy-Chowdhury. Poisson2sparse: Self-supervised poisson denoising from a single image. *arXiv preprint arXiv:2206.01856*, 2022.

[9] Karol Gregor and Yann LeCun. Learning fast approximations of sparse coding. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2010.